Report Progetto S2_L5

Lorenzo Sanna

17 aprile 2026

Obiettivo dell'esercizio

L'esercizio chiede di leggere un programma in Python senza eseguirlo e di trovare gli errori di sintassi, gli errori di logica e le situazioni che il programma non riesce a gestire bene.

Cosa dovrebbe fare il programma

Il programma da come si presenta è un piccolo assistente virtuale. Ogni volta chiede all'utente una domanda e, in base a quello che l'utente scrive, risponde in modo diverso. Se l'utente scrive **"esci"**, il programma saluta e si ferma. Altrimenti usa una funzione che risponde in questo modo:

- se la domanda è "Qual è la data di oggi?" risponde con la data di oggi
- se la domanda è "Che ore sono?" risponde con l'ora attuale
- se la domanda è "Come ti chiami?" risponde "Mi chiamo Assistente Virtuale"
- se la domanda non rientra in nessuno di questi casi, risponde "Non ho capito la tua domanda."

Tabella riassuntiva degli errori

#	Riga	Tipo	Descrizione sintetica
1	7	Sintassi	'break' scritto nel posto sbagliato
2	11	Logico	La funzione viene usata prima di essere scritta
3	13	Sintassi	Nome della funzione scritto male: datetoday()
4	5	Logico	Il confronto distingue maiuscole e minuscole
5	4	Robustezza	Non controlla spazi o testo vuoto
6	12/15/18	Robustezza	La domanda deve essere scritta identica
7	13/16	Stile	Il modulo datetime è usato in modo non uniforme

Analisi dettagliata degli errori

Errore #1 — Riga 7: break indentato male

Tipo: Errore di sintassi

Perché è un errore: la parola **break** non è stata indentata dentro il blocco dell'**if**, quindi sta fuori. In Python non si può mettere un'istruzione tra l'**if** e l'**else**: il programma dà subito errore e non parte. Inoltre, anche se si riuscisse a farlo partire, il **break** verrebbe letto sempre e il ciclo si fermerebbe al primo giro, qualunque cosa scriva l'utente.

Soluzione: spostare **break** un po' più a destra in modo che stia dentro l'**if**, alla stessa altezza di **print("Arrivederci!")**.

Errore #2 — Riga 11: la funzione viene usata prima di essere scritta

Tipo: Errore di logica

Perché è un errore: Python legge il codice dall'alto verso il basso. Alla riga 9 il programma prova a usare la funzione **assistente_virtuale()**, ma la funzione viene scritta solo alla riga 11, quindi più sotto. Quando arriva alla riga 9, Python non sa ancora cos'è **assistente_virtuale** e dà errore.

Soluzione: spostare tutta la definizione della funzione (righe da 11 a 22) sopra il ciclo **while True**, subito dopo **import datetime**. Così quando parte il ciclo, Python ha già letto la funzione e sa come usarla.

Errore #3 — Riga 13: datetime.datetoday() non esiste

Tipo: Errore di sintassi

Perché è un errore: dentro **datetime** non esiste una funzione che si chiama **datetoday()**. È stata scritta male, probabilmente fondendo i nomi di due cose diverse: **date.today()** e **datetime.today()**. Quando Python arriva a questa riga si blocca e dice che il nome non esiste.

Soluzione: scrivere correttamente **datetime.date.today()** per prendere la data di oggi. In alternativa si può usare **datetime.datetime.today()** per essere più coerenti con la riga 16, che usa già **datetime.datetime**.

Errore #4 — Riga 5: il confronto distingue maiuscole e minuscole

Tipo: Situazione non gestita

Perché è un problema: il confronto **comando_utente == "esci"** funziona solo se l'utente scrive esattamente **esci** in minuscolo. Se scrive **Esci**, **ESCI** o mette degli spazi davanti o dietro, il programma non capisce che vuole uscire e risponde "Non ho capito la tua domanda.". Lo stesso problema c'è anche sulle righe 12, 15 e 18.

Soluzione: sistemare il testo scritto dall'utente prima di confrontarlo. Si può usare **.strip()** per togliere gli spazi all'inizio e alla fine, e **.lower()** per mettere tutto in minuscolo. In questo modo **ESCI**, **Esci** e **" esci "** vengono trattati allo stesso modo.

Errore #5 — Riga 4: non vengono controllati gli spazi o il testo vuoto

Tipo: Situazione non gestita

Perché è un problema: se l'utente preme Invio senza scrivere niente, **comando_utente** è vuoto ma il programma lo passa comunque alla funzione e risponde "Non ho capito la tua domanda.". Lo stesso vale se l'utente mette degli spazi prima o dopo la domanda: il testo non corrisponde più a nessuno dei casi previsti.

Soluzione: applicare **.strip()** subito dopo **input()** per togliere gli spazi di troppo.

Errore #6 — Righe 12, 15, 18: le domande devono essere scritte identiche

Tipo: Situazione non gestita

Perché è un problema: il programma riconosce la domanda solo se è scritta **esattamente** come nel codice, compreso il punto interrogativo e l'accento sulla "è". Se l'utente scrive "che ore sono" (senza punto interrogativo), "Che Ore Sono?" (con maiuscole diverse) o "Qual e la data di oggi?" (senza accento), il programma non capisce. Così basta un piccolo errore di battitura per rendere l'assistente inutile.

Soluzione: mettere il testo della domanda in minuscolo con **.lower()** e togliere gli spazi con **.strip()**, poi controllare se contiene certe parole chiave usando l'operatore **in**, invece di confrontare tutta la frase. Per esempio: controllare se nel testo ci sono le parole "data" e "oggi" per la domanda sulla data, la parola "ora" o "ore" per la domanda sull'ora, e "chiami" o "nome" per la domanda sul nome. Così l'assistente capisce anche se la domanda è scritta in modo un po' diverso.

Errore #7 — Righe 13 e 16: datetime usato in modo non uniforme

Tipo: Problema di stile

Perché è un problema: alla riga 13 si cerca di usare **datetime.datetoday()** (che è anche scritto male), mentre alla riga 16 si usa **datetime.datetime.now()**. Usare lo stesso modulo in modi diversi rende il codice più confuso da leggere.

Soluzione: scegliere un solo modo di usare il modulo e tenerlo in tutto il programma. La cosa più pulita è scrivere all'inizio **from datetime import date, datetime**, così si possono usare direttamente **date.today()** e **datetime.now()** senza ripetere ogni volta il nome del modulo.

Conclusioni

Il programma originale ha **due errori** (riga 7 e riga 11) che non lo fanno partire per niente, più un altro errore (riga 13) che si vede solo quando si chiama la funzione. Provando a pensare "fuori dagli schemi", come chiede la traccia, la maggior parte di questi problemi si trova immaginando cosa potrebbe fare un utente vero, che non sa esattamente come è scritto il codice e quindi sbaglia la digitazione. Applicando le soluzioni proposte, il programma dovrebbe funzionare correttamente.